

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería

Ingeniería en Sistemas de Información y Ciencias de la Computación

Curso: Autómatas y Lenguajes Formales

Ing. Nelson Cifuentes

Sección: B



Proyecto Autómatas

Michael Reynaldo Reyes González 0910-24-14419

Jonathan Adrián Herrera Soto 0910-24-15486

Steven David Pérez López 0910-24-17126

03 de octubre, 2026

Índice

1.	Introducción	1
2.	Alcance del Proyecto.....	2
3.	Objetivos	2
3.1	Objetivo General	2
3.2	Objetivos Específicos	2
4.	Fundamento Teórico	3
4.1	Expresiones Regulares	3
4.2	Autómata Finito No Determinista (AFN)	3
4.3	Autómata Finito Determinista (AFD)	3
4.4	Construcción de Thompson.....	4
4.5	Construcción de Subconjuntos (AFN a AFD).....	4
5.	Arquitectura del Sistema y Módulos	5
5.1	Diseño Detallado de Clases.....	5
6.	Análisis Léxico e Implementación en Código.....	6
7.	Interfaz Gráfica de Usuario y Manual de Operación.....	7
7.1	Pestaña 1: Resumen y Postfijo	7
7.2	Pestaña 2: AFN (Thompson).....	8
7.3	Pestaña 3: AFD (Subconjuntos)	9
7.4	Pestaña 4: Probar Cadenas	12
8.	Requisitos, Instalación y Reglas de Sintaxis	13
8.1	Requisitos Previos e Instalación.....	13
8.2	Reglas de Sintaxis para el Usuario	13
9.	Caso de Estudio Completo	14
10.	Solución de Problemas Frecuentes y Limitaciones	15
10.1	Solución de Problemas	15
10.2	Limitaciones y Trabajo Futuro	15
11.	Conclusión.....	16
12.	Evaluación de Desempeño de los Integrantes	17

1. Introducción

El Generador de Autómatas Finitos es un sistema computacional avanzado diseñado con el propósito estricto de procesar una expresión regular para construir automáticamente el Autómata Finito No Determinista (AFN) y su equivalente Autómata Finito Determinista (AFD), además de cadenas con comprobaciones. Desarrollado en el contexto del análisis léxico y la teoría de lenguajes formales, este proyecto materializa conceptos abstractos que son fundamentales para áreas críticas de la informática, tales como el diseño de compiladores, el reconocimiento de patrones y el procesamiento avanzado de texto.

Su finalidad principal radica en permitir la evaluación algorítmica y la comprobación, cadena por cadena, para determinar si un texto específico pertenece de manera exacta al lenguaje formal que dicha expresión describe. Para lograr esta transformación con precisión matemática, el sistema orquesta una serie de algoritmos clásicos: inicia con la conversión de la expresión de notación infija a postfija (apoyándose en el algoritmo de Shunting Yard), aplica el método de construcción de Thompson para generar el modelo no determinista y, finalmente, ejecuta el algoritmo de construcción de subconjuntos para consolidar un autómata determinista eficiente y sin ambigüedades.

2. Alcance del Proyecto

El programa soporta de manera nativa los operadores de unión ($\mid$), cerradura de Kleene ($*$), concatenación implícita y agrupación con paréntesis, que son los requerimientos exigidos por el enunciado del proyecto académico. Además, el sistema incorpora, como extensión propia del equipo de desarrollo, los operadores de una-o-más-veces ($+$) y cero-o-una-vez ($?$). Estas adiciones fueron construidas sobre las mismas reglas del método de Thompson, garantizando que el sistema no se salga del algoritmo formal visto en clase.

3. Objetivos

3.1 Objetivo General

Construir un programa que, a partir de una expresión regular, genere automáticamente su AFN y su AFD, y permita validar si una cadena pertenece al lenguaje descrito por dicha expresión.

3.2 Objetivos Específicos

Implementar el algoritmo de Thompson para transformar una expresión regular en un AFN. Implementar el algoritmo de construcción de subconjuntos para transformar el AFN resultante en un AFD. Simular la ejecución del AFD sobre una cadena ingresada por el usuario, mostrando el recorrido estado por estado. Representar gráficamente cada autómatas generado, tanto en tabla de transiciones como en diagrama de nodos y flechas. Documentar cada etapa de la transformación, de modo que el usuario pueda ver el proceso completo y no solo el resultado final.

4. Fundamento Teórico

Antes de analizar la arquitectura del código, es vital definir los conceptos teóricos fundamentales que rigen la lógica del programa.

4.1 Expresiones Regulares

Una expresión regular (ER) es una notación compacta para describir un conjunto de cadenas (un lenguaje) mediante tres operaciones básicas: concatenación de símbolos, unión de alternativas ($|$) y repetición cero o más veces ($*$).

4.2 Autómata Finito No Determinista (AFN)

Un AFN es una 5-tupla $(Q, \Sigma, \delta, q_0, F)$ donde Q es el conjunto de estados, Σ el alfabeto, δ la función de transición, q_0 el estado inicial y F el conjunto de estados de aceptación. La característica que lo hace "no determinista" es que, desde un mismo estado y con el mismo símbolo, puede haber más de una transición posible, además de las transiciones ϵ (épsilon) que permiten el movimiento sin consumir símbolos.

4.3 Autómata Finito Determinista (AFD)

Un AFD es un caso particular de autómata donde, para cada estado y cada símbolo del alfabeto, existe como máximo una transición, y no existen transiciones ϵ . Esto lo hace directamente ejecutable en sistemas informáticos, ya que en cada paso hay un único estado activo, simplificando la simulación frente a un AFN.

4.4 Construcción de Thompson

El método de Thompson construye el AFN de forma recursiva sobre la estructura de la expresión regular, combinando fragmentos más pequeños mediante transiciones ϵ como "pegamento". Las reglas base implementadas son:

- Símbolo simple a : dos estados unidos por una transición con el símbolo a .
- Concatenación AB : se une el estado final de A con el estado inicial de B mediante una transición ϵ .
- Unión $A|B$: se agregan un nuevo estado inicial (con ϵ hacia A y hacia B) y un nuevo estado final (que recibe ϵ desde el final de A y de B).
- Cerradura de Kleene A^* : se agregan un nuevo estado inicial y uno final; el inicial tiene ϵ hacia A y directo al final, y el final de A regresa con ϵ al inicio de A y avanza con ϵ al nuevo final.
- Toda construcción respeta la propiedad de que cada fragmento generado tiene exactamente un estado inicial y un solo estado de aceptación.

4.5 Construcción de Subconjuntos (AFN a AFD)

La idea central de este algoritmo es que cada estado del AFD representa un conjunto de estados del AFN que podrían estar activos simultáneamente. Se apoya en dos operaciones:

ϵ -clausura(S): dado un conjunto de estados S , devuelve todos los estados alcanzables desde S usando solamente transiciones ϵ .

mover(S , a): dado un conjunto de estados S y un símbolo a , devuelve el conjunto de estados alcanzables consumiendo exactamente ese símbolo desde algún estado de S .

5. Arquitectura del Sistema y Módulos

El programa está organizado en módulos independientes para facilitar las pruebas y el trabajo colaborativo. La arquitectura general fluye desde la expresión regular hacia la notación postfija, luego al generador de Thompson, posteriormente al conversor de subconjuntos, y finalmente al simulador.

5.1 Diseño Detallado de Clases

- **Clase Estado** (estado.py): Representa un nodo del autómata, guardando un identificador numérico único (id) y una bandera esAceptacion.
- **Clase Transicion** (transicion.py): Representa una arista dirigida indicando el estado de origen, el estado de destino y el símbolo (incluyendo ϵ).
- **Clase Automata** (automata.py): Modelo genérico que agrupa estados, transiciones, alfabeto, estado inicial y de aceptación. Funciona como contenedor estructural tanto para el AFN como para el AFD.
- **Clase AnalizadorRegex** (analizador_regex.py): Encargada de convertir la expresión de notación infija a postfija mediante el algoritmo de Shunting Yard. Posee un método insertarConcatenacion que introduce un punto (.) donde la concatenación es implícita.
- **Clase GeneradorThompson** (generador_thompson.py): Construye el AFN leyendo la expresión postfija símbolo por símbolo, manejando una pila de fragmentos de autómata. Posee métodos específicos para crear base, unión, concatenación, estrella de Kleene, y las extensiones de cierre positivo y opcional.
- **Clase ConversorSubconjuntos** (conversor_subconjuntos.py): Implementa el

cálculo de la ϵ -clausura y la función mover. Guarda un registro en texto de los pasos ejecutados para que sean mostrados al usuario.

- **Clase Simulador** (simulador.py): Recibe el AFD (o el AFN) y una cadena para recorrerla símbolo por símbolo. Si logra consumirla por completo y termina en un estado de aceptación, la cadena es aceptada.
- **Clase Graficador** (graficador.py): Utiliza la librería Graphviz para generar diagramas PNG visuales de los autómatas.

6. Análisis Léxico e Implementación en Código

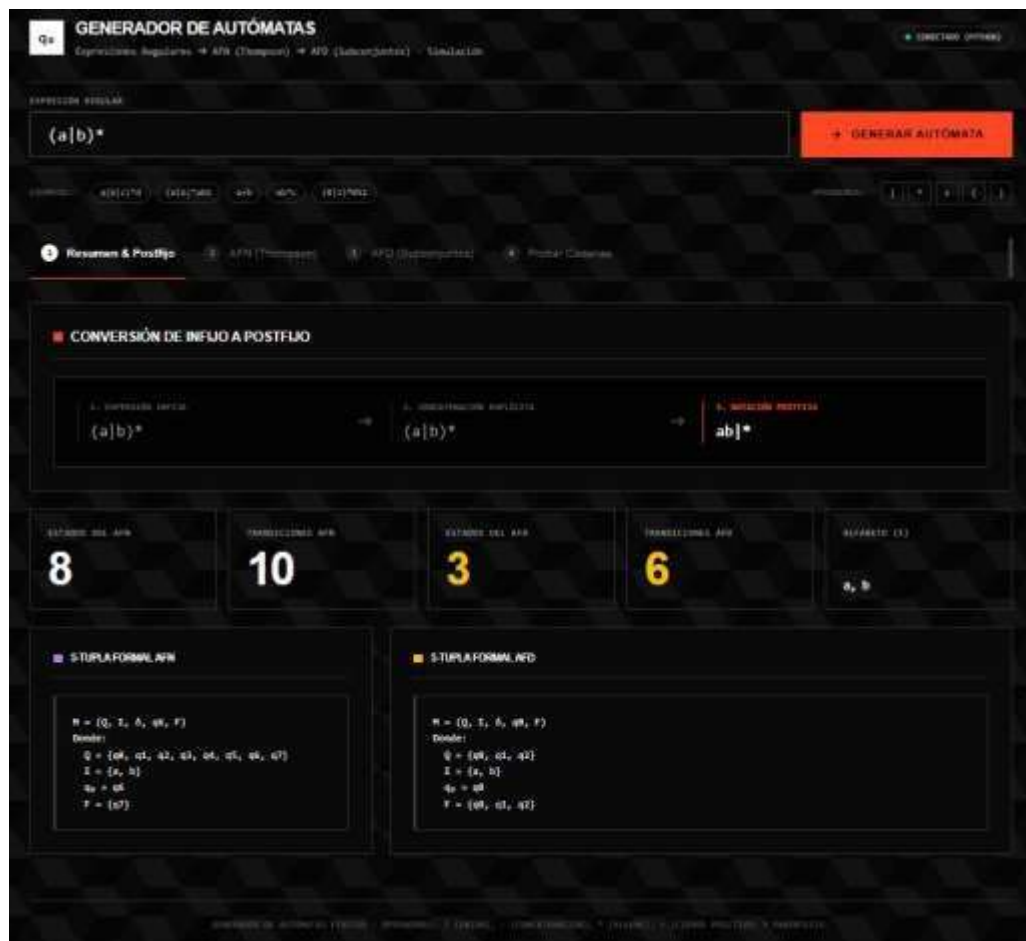
Al examinar el código de AnalizadorRegex, se observa la definición de diccionarios para asignar la precedencia: `OPERADORES_BINARIOS = {"|": 1, ".": 2}` y `OPERADORES_UNARIOS = {"*": 3, "+": 3}`.

El algoritmo evalúa si un carácter es operando mediante la función `esOperando(c)` validando que no sea parte de `|.*(+)`. Si durante la transformación de la expresión se detecta una falta de coincidencia en los paréntesis, el sistema levanta de inmediato una excepción del tipo `ValueError` indicando "Paréntesis desbalanceados en la expresión regular".

7. Interfaz Gráfica de Usuario y Manual de Operación

7.1 Pestaña 1: Resumen y Postfijo

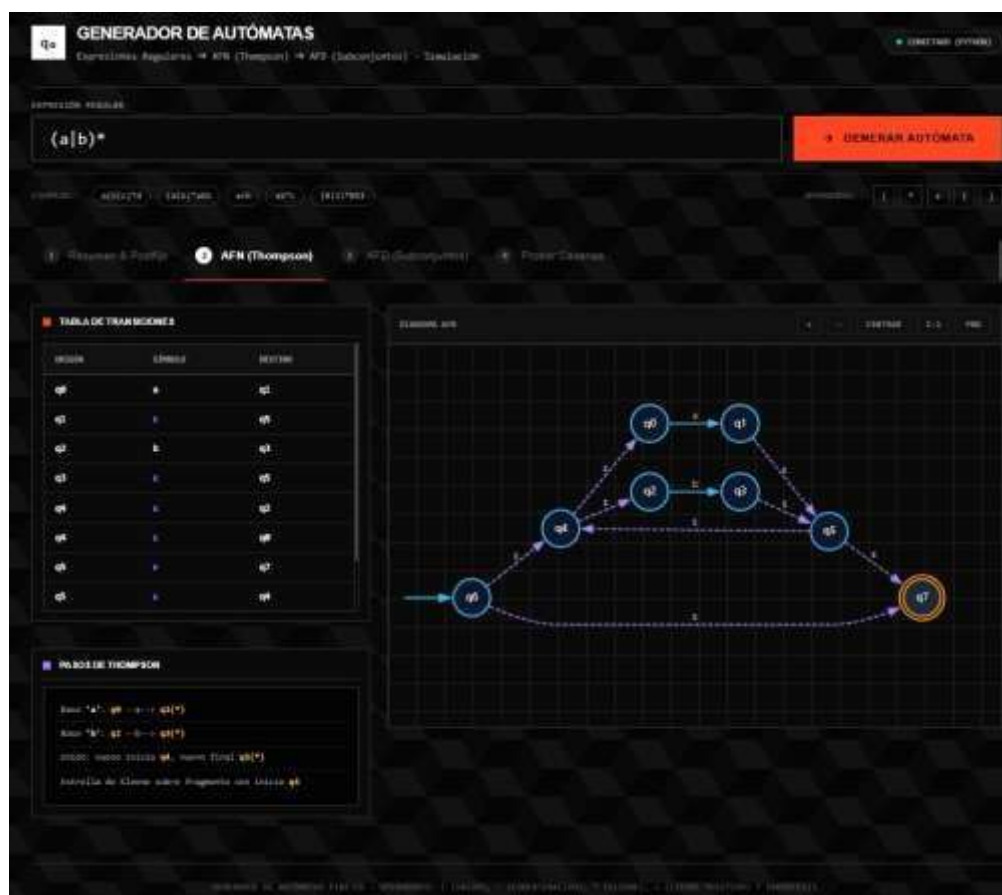
En esta vista inicial, el usuario ingresa la expresión regular en el campo de texto superior y presiona el botón "Generar Autómata" (ver Figura 1). De manera inmediata, el sistema procesa la cadena y despliega en la sección inferior la transformación léxica hacia la notación postfija. Asimismo, presenta un resumen estadístico que detalla la cantidad de estados y transiciones generadas tanto para el AFN como para el AFD, incluyendo el alfabeto detectado. En la parte final de esta pantalla, se exhibe la definición matemática de la 5-tupla (estados, alfabeto, función de transición, estado inicial y estados de aceptación) correspondiente a cada autómata.



7.2 Pestaña 2: AFN (Thompson)

Esta sección está dedicada a la visualización del Autómata Finito No Determinista. La pantalla se divide en dos áreas principales:

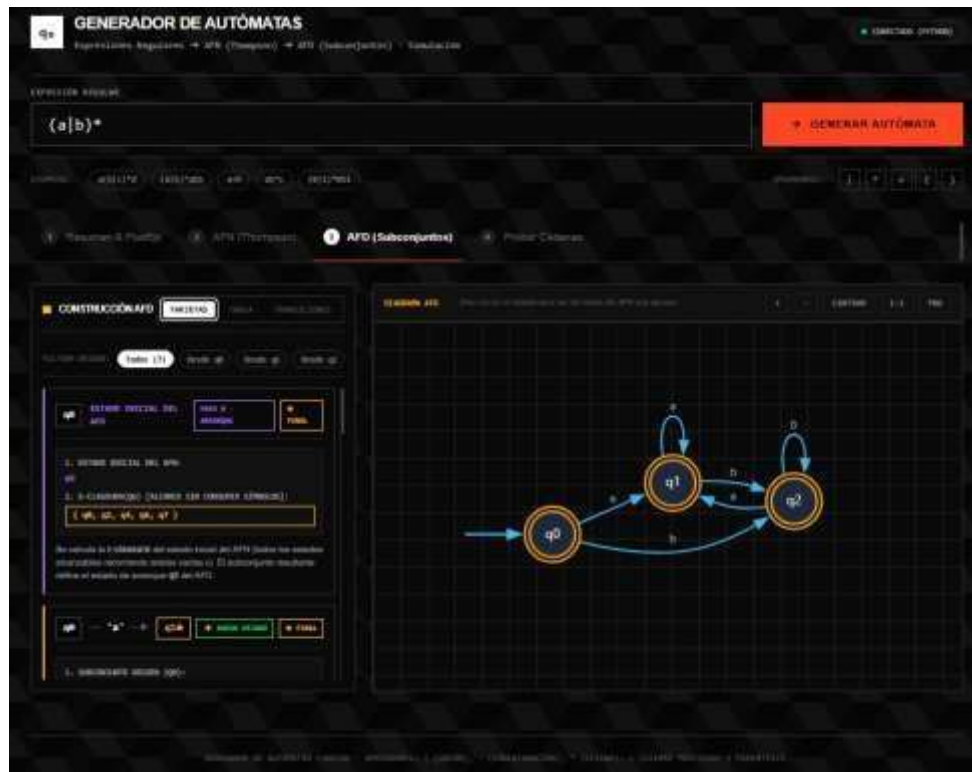
- **Tabla de Transiciones:** Muestra un listado detallado donde cada fila indica el estado de origen, el símbolo consumido (incluyendo transiciones vacías o épsilon, representadas con ϵ) y el estado de destino.
- **Diagrama Interactivo:** Presenta el grafo visual del AFN. El usuario puede manipular esta vista utilizando herramientas de acercamiento (zoom in/out), el botón "Centrar" para ajustar el grafo a la pantalla, o la opción "1:1" para restaurar la escala original. Adicionalmente, cuenta con un botón "PNG" que permite exportar y descargar el diagrama generado.



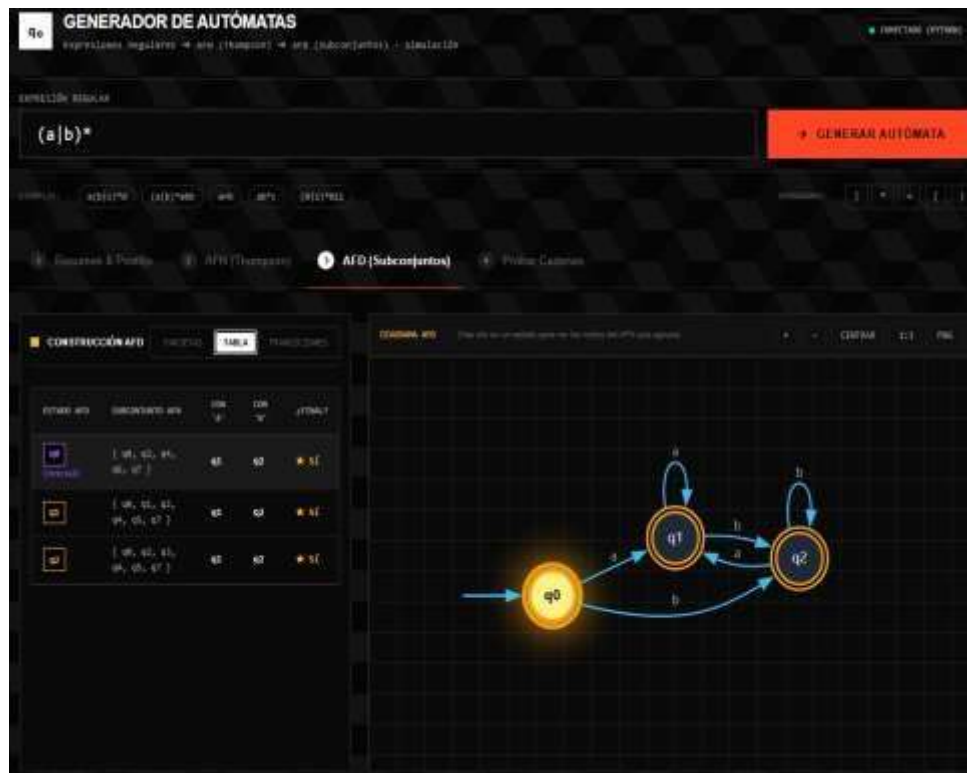
7.3 Pestaña 3: AFD (Subconjuntos)

Esta vista expone el resultado del algoritmo de subconjuntos, mostrando el autómata determinista minimizado. El grafo interactivo mantiene las mismas herramientas de manipulación (zoom, centrar, exportar a PNG) que en la pestaña anterior, pero el panel lateral izquierdo ofrece tres modos de análisis profundo:

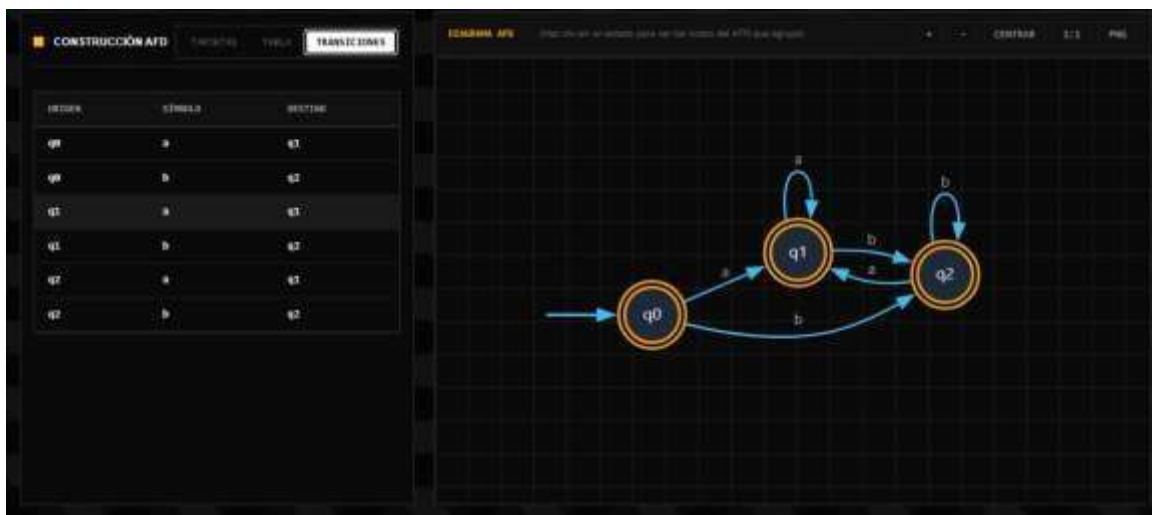
- **Tarjetas Explicativas:** Al posicionar el cursor o hacer clic sobre un nodo específico en el diagrama, el panel izquierdo despliega una tarjeta que explica detalladamente el proceso matemático (como el cálculo de la ϵ -clausura) que originó dicho estado determinista a partir de los nodos del AFN.



- Subconjuntos: Muestra la relación directa y la agrupación de estados del AFN que conforman cada nuevo estado del AFD.



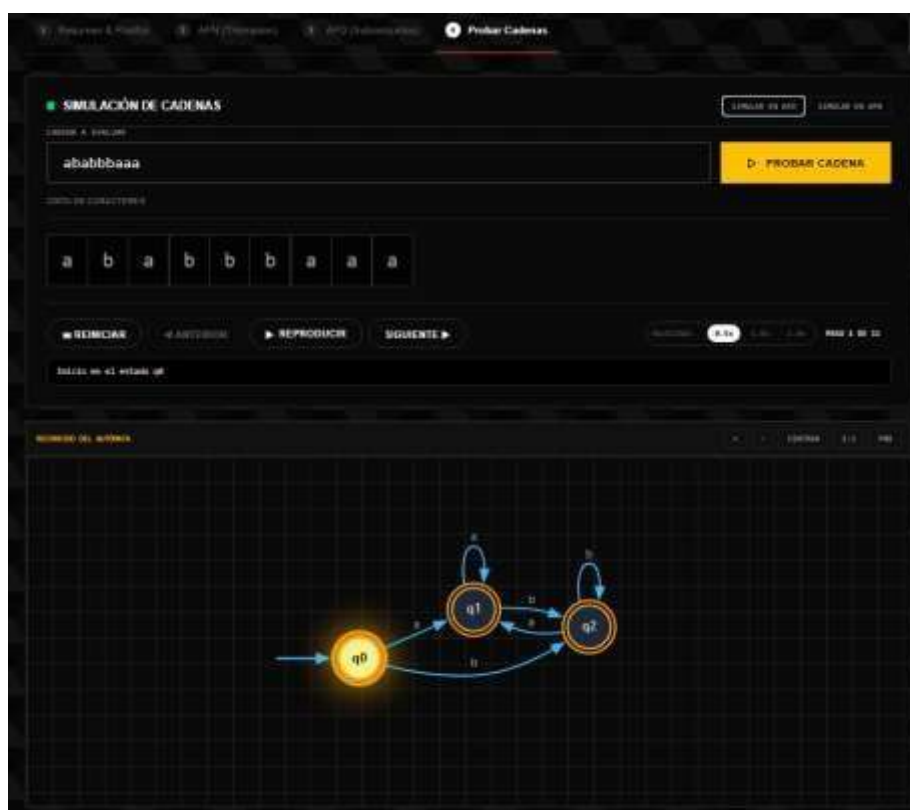
- **Tabla de Transiciones:** Presenta de manera clara y definitiva las transiciones consolidadas del autómata determinista, indicando el estado de origen, el símbolo y el estado de destino, sin ambigüedades.

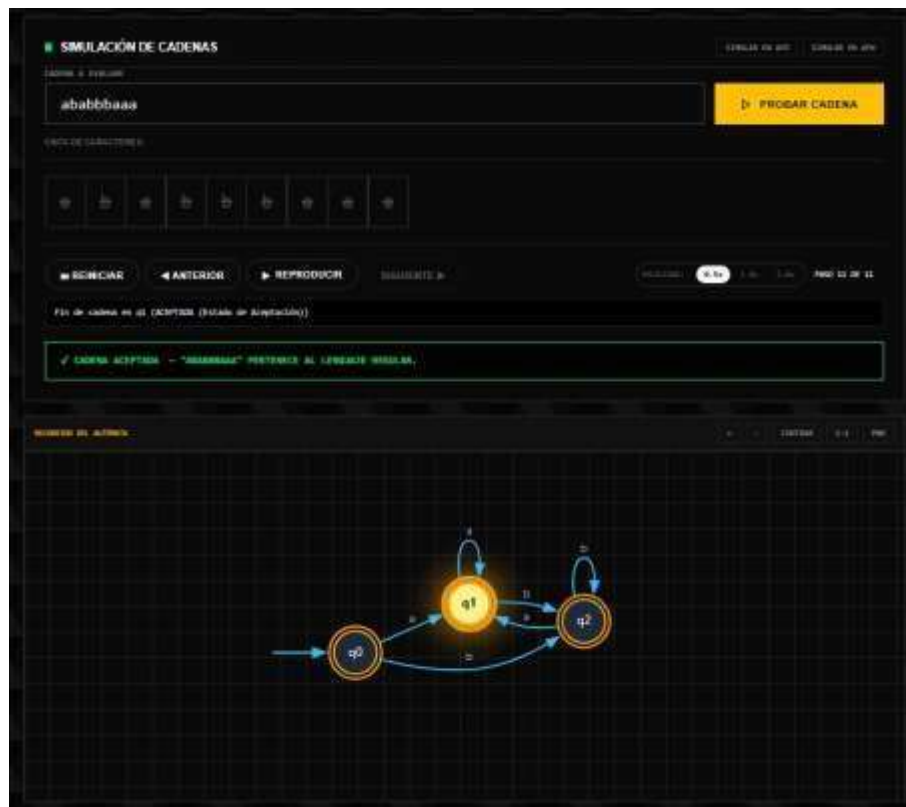


7.4 Pestaña 4: Probar Cadenas

El módulo final permite validar cadenas de texto contra el lenguaje regular definido. El usuario ingresa una cadena de prueba y ejecuta la validación paso a paso o de manera automática utilizando el botón "Reproducir". Durante la simulación automática, es posible ajustar la velocidad de ejecución mediante un selector lateral (con opciones de 0.5x, 1.0x o 2.0x).

Conforme avanza la lectura de la cadena, el diagrama interactivo ilumina secuencialmente los nodos y transiciones activos, brindando una retroalimentación visual clara del recorrido del autómata. Al finalizar, si el último símbolo consumido posiciona al autómata en un estado de aceptación, el sistema emite un mensaje en color verde confirmando que la cadena es válida; por el contrario, si el proceso se detiene en un estado de rechazo o no existe una transición válida, se despliega una alerta en color rojo indicando que la cadena no pertenece al lenguaje.





8. Requisitos, Instalación y Reglas de Sintaxis

8.1 Requisitos Previos e Instalación

- Se requiere Python 3.8 o una versión superior.
- Es obligatorio instalar el binario del programa Graphviz en el sistema operativo, configurándolo en el PATH, y su respectiva librería de Python ejecutando `pip install graphviz`.

8.2 Reglas de Sintaxis para el Usuario

- **Concatenación implícita:** Expresar un carácter tras otro (ej. `ab`).
- **Unión (`|`):** Representa una opción u otra (ej. `a|b`).

- **Cerradura de Kleene (*):** Admite cero o múltiples repeticiones de la expresión antecedente.
- **Cierre Positivo (+):** Admite una o más repeticiones, pero no acepta la cadena vacía.
- **Cierre Opcional:** Exige cero o exactamente una repetición del operando.
- El sistema es sensible a mayúsculas y minúsculas ("Case Sensitive"), por lo que "a" es completamente diferente de "A".

9. Caso de Estudio Completo

Para demostrar el flujo completo del sistema se utiliza la expresión de prueba: $a(b|c)^*d$.

- **Notación:** La cadena original pasa a $a.(b|c)^*.d$ y su forma postfija finaliza en $abc|^*.d$.
- **AFN:** El algoritmo procesa a, b, y c como reglas base creando fragmentos de dos estados cada uno. Aplica la unión sobre b y c (agregando un nuevo inicio q6 y un final q7). Luego aplica la estrella de Kleene, y concatena con los demás fragmentos, resultando en un AFN de 12 estados (desde q0 hasta q11) donde q11 es el estado de aceptación.
- **AFD:** Al iniciar, se calcula la ϵ -clausura del estado inicial, resultando en $\{q0\}$. Al evaluar la función mover y calcular clausuras adicionales con el alfabeto $\{a, b, c, d\}$, el sistema consolida los estados que pueden estar activos simultáneamente (como el solapamiento de ramas b y c). El proceso concluye generando un autómata compacto de tan solo 5 estados, demostrando el éxito matemático del algoritmo de subconjuntos.
- **Simulación:** Al evaluar la cadena "abccbd", el programa avanza de q0 a q1 con 'a', a q2 con 'b', a q3 con 'c', manteniéndose en q3 o retornando a q2 de forma iterativa, y finalmente consumiendo 'd' para terminar en q4. Dado que q4 es un estado de

aceptación, la cadena es aceptada exitosamente. Si se ingresa "aed", el autómata rechaza inmediatamente al no encontrar una transición para el símbolo 'e'.

10. Solución de Problemas Frecuentes y Limitaciones

10.1 Solución de Problemas

- **ModuleNotFoundError:** Indica que la librería Python no está instalada; se soluciona ejecutando `pip install graphviz`.
- **ExecutableNotFound:** Advierte que Graphviz no está en el PATH del sistema; requiere instalar el programa base y reiniciar la terminal.
- **Error por símbolo no soportado:** Sucede al emplear caracteres con significado especial no reconocidos (como espacios o corchetes).

10.2 Limitaciones y Trabajo Futuro

- No es posible utilizar símbolos reservados matemáticos (como el asterisco o los paréntesis) como parte literal del alfabeto a analizar, ya que no se ha implementado un sistema de escapes o secuencias literales.
- Las expresiones regulares vacías dentro de paréntesis, como `()`, generan errores controlados en código pero carecen de una retroalimentación específica y amigable hacia el usuario.

11. Conclusión

La fragmentación del proyecto en cuatro etapas modulares (análisis léxico, algoritmo de Thompson, construcción de subconjuntos y simulación) asegura un diseño computacional estable y desacoplado. Esta separación de responsabilidades optimiza el trabajo colaborativo y garantiza que el procesamiento de las expresiones regulares se realice con alta eficiencia, permitiendo modificaciones aisladas sin afectar la integridad del sistema global.

Asimismo, la verificación cruzada entre las simulaciones del AFN y el AFD brinda una prueba irrefutable sobre la precisión de los algoritmos implementados. A pesar de la drástica reducción de estados estructurales lograda mediante la construcción de subconjuntos, el lenguaje formal reconocido se mantiene matemáticamente inalterable, lo cual valida de forma práctica el éxito teórico e informático del proyecto.

Finalmente, la incorporación de una interfaz gráfica interactiva consolida al sistema como una solución integral y un valioso instrumento educativo. Al proyectar de manera visual, dinámica y en tiempo real todo el flujo de transformaciones lógicas, el programa supera el alcance de un evaluador de consola tradicional y cumple a cabalidad con los objetivos académicos al aplicar la teoría de lenguajes formales en un entorno de desarrollo moderno.

12. Anexos

12. Evaluación de Desempeño de los Integrantes

Formulario de Evaluación entre Pares

Nombre del evaluador: Michael Reynaldo Reyes

González

Nombre del compañero evaluado: Steven David Pérez

López

Instrucciones: Marque con una **X** el nivel que mejor describe el desempeño de su compañero. Esta información será confidencial y utilizada para asignar un punto individual.

Criterio	Excelente (1.0)	Bueno (0.75)	Regular (0.5)	Deficiente (0.25)
Cumplió con las tareas asignadas a tiempo	X	☐	☐	☐
Participó activamente en las reuniones y discusiones	X	☐	☐	☐
Aportó ideas y soluciones relevantes al equipo	X	☐	☐	☐
Colaboró con actitud respetuosa y constructiva	X	☐	☐	☐

Formulario de Evaluación entre Pares

Nombre del evaluador: Michael Reynaldo Reyes

González

Nombre del compañero evaluado: Jonathan Adrián

Herrera Soto

Instrucciones: Marque con una **X** el nivel que mejor describe el desempeño de su compañero. Esta información será confidencial y utilizada para asignar un punto individual.

Criterio	Excelente (1.0)	Bueno (0.75)	Regular (0.5)	Deficiente (0.25)
Cumplió con las tareas asignadas a tiempo	X	☐	☐	☐
Participó activamente en las reuniones y discusiones	X	☐	☐	☐
Aportó ideas y soluciones relevantes al equipo	X	☐	☐	☐
Colaboró con actitud respetuosa y constructiva	X	☐	☐	☐

Formulario de Evaluación entre Pares

Nombre del evaluador: Jonathan Adrián Herrera Soto

Nombre del compañero evaluado: Michael Reynaldo

Reyes González

Instrucciones: Marque con una **X** el nivel que mejor describe el desempeño de su compañero. Esta información será confidencial y utilizada para asignar un punto individual.

Criterio	Excelente (1.0)	Bueno (0.75)	Regular (0.5)	Deficiente (0.25)
Cumplió con las tareas asignadas a tiempo	X	☐	☐	☐
Participó activamente en las reuniones y discusiones	X	☐	☐	☐
Aportó ideas y soluciones relevantes al equipo	X	☐	☐	☐
Colaboró con actitud respetuosa y constructiva	X	☐	☐	☐